

Semana 10 — Inventário e ampliação do Interaction System

Semana 10

Inventário e ampliação do Interaction System

Disciplina: Tendências de Motores de Jogos (IN46A)

Curso: Tecnologia em Jogos Digitais — IFMS Campus Dourados

Unidade III — Resolver Problemas (Semanas 8–11)

Projeto: Vertical Slice *O Templo Esquecido*

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

- Compreender padrões de Inventory System — armazenamento, adição/remoção, exibição — como problema estrutural comum a qualquer engine
- Estruturar um `InventoryComponent` reutilizando diretamente o `ItemData` já modelado na Semana 6, sem duplicar dados de item
- Retomar e ampliar o contrato `Interactable` da Semana 5 para múltiplos tipos de interação
- Conectar o Interaction System ampliado ao inventário (coletar, usar, descartar), propondo com autonomia própria um novo tipo de interação
- Submeter os sistemas de inventário e interação a Code Review formal (Rubrica 4)

Semana 10 — Inventário e ampliação do Interaction System

ENCONTRO 1

InventoryComponent: Armazenar o que já existe

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 1

- Revisão do Encontro 2 da Semana 9 (HUD funcional sobre CanvasLayer) (15 min)
- Introdução: os três problemas de todo Inventory System (20 min)
- Demonstração: estruturação guiada de um `InventoryComponent` reutilizando `ItemData` já existente (30 min)
- Laboratório: cada grupo estrutura seu próprio `InventoryComponent`, conectando-o à coleta já existente (55 min)
- Feedback e fechamento (15 min)



Os itens já existem como `ItemData`. Para onde vão quando o jogador os coleta?

Pense no que acontece hoje, no projeto, quando um `Chest` ou `Pickup` emite `item_collected` — o handler só guarda o nome do item.

O Problema: Onde Vive a Posse do Item?

- `ItemData` (Resource + Enum, desde a Semana 6) define nome, ícone e tipo do item — mas não diz quem o possui
- Hoje o handler de coleta só registra o `nome` do item em `SaveManager.itens_coletados` (String) — o `ItemData` completo não é guardado em lugar nenhum
- Falta um lugar que armazene, liste e permita manipular os itens que o jogador possui agora

Três Problemas de Todo Inventory System

- **Armazenamento** — onde vive a lista de itens que o jogador possui
- **Adição/remoção** — como itens entram e saem dessa lista ao longo do jogo
- **Exibição** — como esse estado se torna visível ao jogador (resolvido no Encontro 2, sobre os princípios de Control node da Semana 9)

InventoryComponent: Mesmo Padrão de Component

- Novo Component do Player, seguindo o mesmo padrão de composição de `InteractionComponent`, `HealthComponent` e `SaveComponent`
- Armazena referências a instâncias de `ItemData` já modeladas — nunca duplica ou recria seus dados
- Reforça o princípio de composição ensinado desde a Semana 4 (`GameManager`): cada responsabilidade isolada em um Component reutilizável

Armazenamento de Itens no Godot × Unity

Godot

- `ItemData` (Resource) define o item
- `InventoryComponent` (Node filho do Player) armazena as instâncias possuídas

Unity

- `ScriptableObject` define o item — equivalente direto ao `ItemData`
- `MonoBehaviour` de inventário (ou `ScriptableObject` de runtime set) armazena as instâncias possuídas

Demonstração — InventoryComponent Guiado

O que será construído:

- Um `InventoryComponent` capaz de adicionar, remover e consultar `ItemData` já modelados
- Conexão à coleta já existente via `Chest` / `Pickup`

Por quê: dar a cada grupo a base direta para a `InventoryUI` do Encontro 2, sem duplicar dados de item.

Imagem sugerida

*Objetivo: mostrar o **InventoryComponent** armazenando referências a **ItemData** sem duplicar seus dados.*

Enquadramento: árvore de cena do Player ao lado de uma coleção de recursos.

*Elementos presentes: "Player" com filhos "InteractionComponent", "HealthComponent" e "InventoryComponent"; setas do **InventoryComponent** apontando para instâncias de **ItemData** já existentes (não para cópias).*

*Destaque visual: a seta de referência, contrastando com uma alternativa errada (dados de item recriados dentro do **InventoryComponent**) marcada com um X.*

Legenda sugerida: "O InventoryComponent referencia o ItemData — nunca recria seus dados."

Arquitetura — Do Chest/Pickup ao Inventário

- O handler do Signal `item_collected` (Semana 7) passa a repassar o `ItemData` ao `InventoryComponent` — `Chest / Pickup` não são alterados
- O item some do mundo e passa a existir como entrada no inventário — evento único, sem duplicação
- Nenhum sistema de coleta é reescrito: o `InventoryComponent` se conecta ao que já existe

Laboratório — InventoryComponent Funcional

Cada grupo estrutura, no próprio projeto:

1. Um `InventoryComponent` com métodos de adicionar, remover e consultar `ItemData`
2. Conexão do `InventoryComponent` à coleta já existente via `Chest` / `Pickup`
3. Verificação de que o item é removido da cena ao ser adicionado ao inventário

Fechamento — Encontro 1

- `InventoryComponent` funcional, armazenando os `ItemData` coletados via `Chest / Pickup`
- Adicionar, remover e consultar itens sem duplicar dados de definição
- Nenhuma alteração no HUD ou em outro sistema de gameplay da Semana 9
- Próximo passo: ampliação do contrato `Interactable` e Code Review, no Encontro 2

Semana 10 — Inventário e ampliação do Interaction System

ENCONTRO 2

Ampliando o Interaction System

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 2

- Revisão do Encontro 1 (InventoryComponent funcional) (15 min)
- Demonstração: ampliação guiada do contrato Interactable , conectado ao inventário (25 min)
- Apresentação do desafio: empilhar, combinar ou interação com cooldown (15 min)
- Laboratório/Desafio: conexão inventário-interação e o novo tipo de interação (50 min)
- Code Review formal (Rubrica 4) (30 min)

Semana 10 — Inventário e ampliação do Interaction System



O contrato `Interactable` já resolveu portas e alavancas. Ele precisa ser reescrito para coletar, usar e descartar itens?

Pense no que já existe desde a Semana 5, antes de propor algo novo.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

O Problema: Conectar Inventário e Ação no Mundo

- O Encontro 1 resolveu o armazenamento de itens
- Falta conectar esse armazenamento à ação do jogador no mundo — coletar, usar, descartar
- O contrato `Interactable` (Semana 5) já resolve comunicação desacoplada; a ampliação apenas o aplica a múltiplos tipos simultâneos

Retomando o Contrato Interactable

- Fundamentado na Semana 5 para casos simples (`Door` , `Lever`): detecção via `InteractionComponent` / `Area3D`, resposta via `has_method` / `Signals`
- Ampliação: múltiplos tipos de interação exigem **priorização** (qual interativo responde quando mais de um está ao alcance)
- Ampliação: resposta **diferenciada** conforme o tipo — coletar não é a mesma ação que usar ou descartar

Conectando ao InventoryComponent

- **Coletar** — adiciona o item ao `InventoryComponent`
- **Usar** — consome ou aplica o efeito de um item já no inventário
- **Descartar** — remove do inventário e reintroduz o item no mundo

Ampliação de Interação no Godot × Unity

Godot

- Mesmo contrato `Interactable` (`has_method` /duck typing), estendido com tipo de interação
- Priorização entre múltiplos interativos resolvida no `InteractionComponent`

Unity

- Mesma interface C# (ex.: `IInteractable`) estendida, ou interfaces adicionais (`ICollectible` , `IUsable` , `IDiscardable`)
- Priorização tipicamente por distância ou `SphereCollider` / `OverlapSphere` , equivalente ao `Area3D`

Demonstração — Interação Ampliada e Conectada

O que será construído:

- Contrato `Interactable` ampliado para múltiplos tipos, com priorização entre interativos ao alcance
- Conexão de coletar/usar/descartar ao `InventoryComponent` do Encontro 1

Por quê: fixar a ampliação guiada antes de abrir o desafio de um novo tipo de interação.

Imagem sugerida

Objetivo: mostrar a priorização entre múltiplos interativos detectados simultaneamente pelo

***InteractionComponent**.*

Enquadramento: cena de jogo em planta baixa, Player ao centro.

*Elementos presentes: Player com raio de alcance do **InteractionComponent**, dois interativos dentro do alcance (um **Chest**, um **NPC**), com destaque no interativo priorizado.*

Destaque visual: o interativo priorizado em cor sólida, o não priorizado esmaecido.

Legenda sugerida: "A decisão de qual interativo responde pertence ao InteractionComponent, não a cada Interactable."

Arquitetura — Contrato Estendido, Não Reescrito

- `ItemData` (Semana 6) e o contrato `Interactable` (Semana 5) continuam sendo os mesmos, apenas reutilizados em um novo cenário
- A priorização entre interativos é responsabilidade do `InteractionComponent`, nunca de cada `Interactable` individualmente
- Nenhuma lógica de coleta/uso/descarte deve duplicar o que já existe no `InventoryComponent`



Desafio — Novo Tipo de Interação

Expanda seu sistema de interação para suportar um novo tipo — **empilhar itens, combinar itens** ou **interação com cooldown** — mantendo a comunicação desacoplada via contrato `Interactable`.

OBJETIVOS

Entrega: Code Review (Rubrica 4) — organização, modularidade, reutilização de `ItemData` e do contrato `Interactable`, comunicação desacoplada entre sistemas.

Boas Práticas — Extensão sem Duplicação

- Estender o contrato `Interactable` existente, nunca reescrevê-lo do zero para um novo caso
- Concentrar a priorização entre interativos no `InteractionComponent`, não em cada `Interactable`
- Reutilizar a lógica já existente no `InventoryComponent` ao implementar o novo tipo de interação, sem duplicá-la

Code Review — Rubrica 4

- Organização dos scripts/Orchestrations e nomenclatura
- Modularidade e reutilização de `ItemData` (Semana 6) e do contrato `Interactable` (Semana 5)
- Comunicação desacoplada entre sistemas
- Conduzido como diálogo técnico: o próprio grupo explica sua lógica ao professor

Fechamento — Encontro 2

- `InventoryComponent` conectado ao Interaction System ampliado — coletar, usar, descartar
- Novo tipo de interação (empilhar, combinar ou cooldown) proposto e implementado com solução própria
- Sistemas submetidos a Code Review formal (Rubrica 4), sem duplicação de lógica entre `Chest` / `Pickup` / `InventoryComponent`

Resultado Esperado da Semana

- `InventoryComponent` funcional armazenando os `ItemData` coletados, com `InventoryUI` inicial exibindo esses dados
- Interaction System ampliado conectando coletar, usar e descartar itens ao inventário
- Novo tipo de interação (empilhar, combinar ou cooldown) proposto e implementado com solução própria
- Conjunto submetido a Code Review formal (Rubrica 4)

Checklist da Semana

- [] `InventoryComponent` armazenando `ItemData` coletados via `Chest` / `Pickup`, sem duplicar dados de definição
- [] Contrato `Interactable` ampliado para múltiplos tipos, com priorização no `InteractionComponent`
- [] Coletar, usar e descartar conectados ao `InventoryComponent`
- [] Novo tipo de interação (empilhar, combinar ou cooldown) implementado com solução própria
- [] Code Review (Rubrica 4) concluído, sem duplicação de lógica entre sistemas

Próximos Passos — Semana 11

O `HealthComponent` (Semana 8) e o Interaction System ampliado desta semana são pré-requisitos diretos da Semana 11, que encerra o Módulo 3 (Unidade III) introduzindo `NavigationRegion3D/NavigationAgent3D`, Behavior Tree e Blackboard via LimboAI para dar autonomia a um `Enemy`, além de um combate simples (`Area3D/RayCast3D` chamando `apply_damage` no `HealthComponent` do `Enemy`). A Semana 11 encerra a Unidade III, concentrando os instrumentos avaliativos que fecham o módulo.

Leitura recomendada: Godot Docs — Resources, Nodes and Scene Instances, Signals; Unity Manual (consulta comparativa) — ScriptableObject.